LESSONS LEARNED REPORT

Post-Simulation Review — Registry Run Key Persistence Detection

---

Report ID: LL-2026-001 **Simulation Reference:** IR-2026-001 **Date:** May 5, 2026 **Analyst:** Olayinka Oyetade **Review Type:** Post-Incident / Post-Simulation Analysis

---

1. SIMULATION SUMMARY

This report documents the lessons learned from building and executing a simulated Registry Run Key persistence attack in a home SOC lab environment. The goal was to understand how attackers establish persistence on Windows endpoints, and how SOC analysts detect and respond to that behavior using Sysmon and Splunk.

What Worked: End-to-end detection pipeline from attack execution to automated email alert in under 5 minutes.

What Was Challenging: Query refinement — early SPL queries returned too many results (false positives) from legitimate Windows processes also interacting with registry Run keys.

---

2. TECHNICAL LESSONS

2.1 Behavioral Detection Outperforms Signature Detection

What I learned: Early in the simulation, the idea was to detect the specific file `malware.exe`. This approach is fundamentally flawed — in a real attack, the attacker would simply rename the file or use a different payload. The correct approach is detecting the **behavior**: the act of a scripting engine (PowerShell) modifying a persistence registry location.

Real-world application: Production SOC rules at enterprise organizations are built around behaviors (MITRE ATT&CK techniques), not signatures. A signature blocks one malware sample. A behavioral rule blocks the entire technique — regardless of the specific tool or

payload used.

Example: The detection query `EventCode=13 "powershell.exe" "CurrentVersion\\Run"` would catch Emotet, TrickBot, a custom C2 framework, or a red team operator — all with the same single rule. It doesn't matter what the malware is called or what it does. The persistence behavior is identical.

2.2 False Positives Are a Real Problem — And Must Be Engineered Against

What I learned: The first version of the detection query returned results for legitimate Windows processes that also write to Run keys (Windows installers, OneDrive, certain Microsoft updates). These are false positives — real alerts that are actually benign.

The problem with false positives: In production, false positive fatigue is one of the biggest challenges SOC analysts face. If an alert fires 200 times a day and 195 of them are benign, analysts start ignoring the alert entirely — including the 5 real incidents. This is called "alert fatigue" and it's a leading cause of SOC failures.

How I solved it: Refined the query to filter by source process. Legitimate software typically writes to Run keys via their own installer processes (msiexec.exe, setup.exe). Malware typically uses PowerShell, cmd.exe, wscript.exe, or other scripting engines. Adding `Image="*powershell.exe"` to the query dramatically reduced false positives while preserving high-fidelity detections.

Real-world application: Production detection engineering always involves an iterative tuning cycle:

1. Write detection rule
2. Deploy and observe results
3. Identify false positives
4. Add exclusions
5. Repeat until signal-to-noise ratio is acceptable

This simulation replicated that real process.

2.3 Sysmon Configuration Is Critical — Garbage In, Garbage Out

What I learned: Sysmon must be configured correctly before it will generate useful telemetry. Without the correct XML configuration file, Sysmon either generates too much

data (logging everything, making analysis impossible) or misses critical events entirely.

Key insight: Event ID 13 (Registry Value Set) does not fire by default. It must be explicitly enabled in the Sysmon configuration XML with the correct `TargetObject` filters. Without this, the entire detection pipeline fails silently — the attack happens, but no telemetry is generated.

Real-world application: In production environments, Sysmon configuration management is a dedicated responsibility. Organizations like SwiftOnSecurity have published open-source Sysmon configuration files used widely across the industry. Understanding why Sysmon needs configuration — not just that it does — makes you a stronger candidate.

2.4 HKCU vs HKLM — Privilege Escalation Context Matters

What I learned: The simulation used `HKCU` (HKEY_CURRENT_USER) for the registry key, not `HKLM` (HKEY_LOCAL_MACHINE). This distinction matters for investigation:

Registry Hive	Scope	Requires Admin?	Persistence Type
HKCU	Current user only	No	User-level persistence
HKLM	All users / system-wide	Yes	System-level persistence

Real-world application: If an attacker has only user-level access (no privilege escalation), they can only write to HKCU — persistence only survives that specific user's login. If they've escalated to admin, they write to HKLM — persistence survives any user login. Detecting HKLM modifications is a higher-severity finding because it implies privilege escalation has already occurred.

2.5 End-to-End Pipeline Validation Is Non-Negotiable

What I learned: It was tempting to declare success after seeing logs in Splunk. But the real validation was confirming the full pipeline: attack → Sysmon capture → Splunk ingestion → detection rule match → automated alert → analyst notification. Each link in the chain can silently fail.

Real-world application: Production SOC teams regularly run "purple team" exercises where the red team (attackers) executes known techniques specifically to verify that detection tools are working end-to-end. A detection rule that exists in Splunk but is never tested may fail silently for months before anyone notices.

---

3. OPERATIONAL LESSONS

3.1 Documentation Is Part of the SOC Job

Real SOC analysts spend a significant portion of their time documenting — incident reports, playbooks, detection logic, lessons learned. This simulation produced 4 separate professional documents. That is realistic for a real SOC incident.

3.2 The Attacker Has Time — The Defender Must Have Automation

This simulation demonstrated the importance of automated alerting. If detection required manual Splunk queries every hour, the attack could persist for hours or days. Automated detection with a 5-minute alert window dramatically reduces Mean Time to Detect (MTTD) — a key SOC KPI.

3.3 Context Transforms Data Into Intelligence

Raw logs are data. Knowing that `EventCode=13` from `powershell.exe` targeting `CurrentVersion\Run` with a path pointing to `C:\Users\Public\` is a high-confidence persistence indicator — that is intelligence. The difference between a junior and senior analyst is often the ability to interpret context, not just read logs.

---

4. WHAT I WOULD DO DIFFERENTLY

Gap	Improvement
Snapshot the VM before and after attack	Enables before/after comparison for investigation
Enable more Sysmon event types	Event ID 1 (Process Create) and Event ID 11 (File Create) would enrich the investigation
Simulate HKLM modification for comparison	Demonstrates privilege escalation context
Add a second attacker machine	Would test network-based detection, not just host-based
Build a Splunk dashboard	Visual correlation of multiple events is more realistic than raw tables

---

5. KEY TAKEAWAYS FOR INTERVIEWS

> "The most valuable insight from this simulation was learning the difference between detection and good detection. The first query I wrote caught the attack — but it also caught dozens of legitimate processes, which would have created alert fatigue in a real SOC environment. Refining the rule to filter by source process (PowerShell) improved signal-to-noise ratio dramatically. That tuning process is something I now understand is a core part of detection engineering in production SOC environments."

Lessons Learned Report prepared by: Olayinka Oyetade | SOC Lab Post-simulation analysis for portfolio documentation